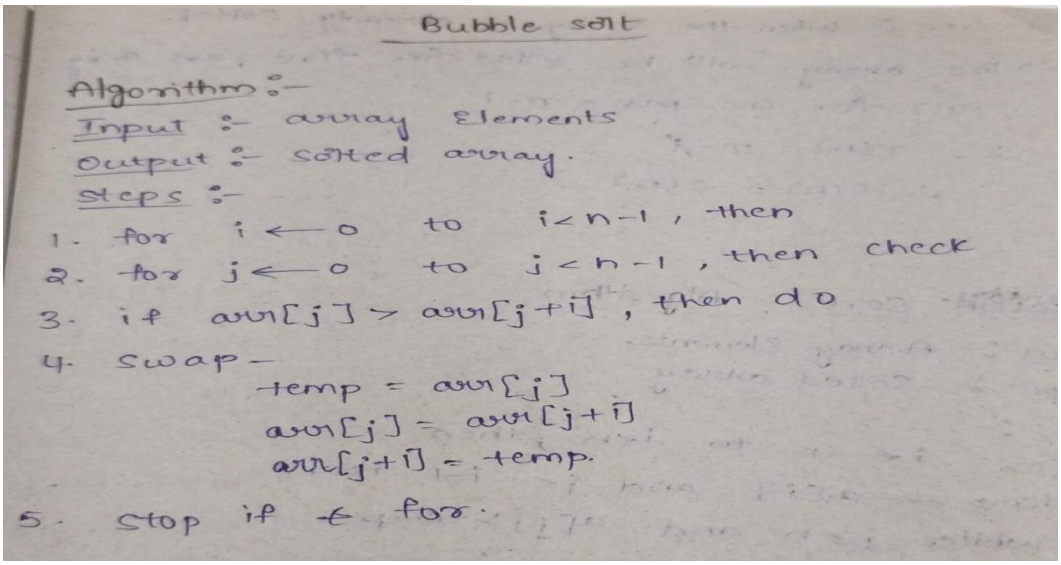
Practical - 1

Aim: To Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort

Bubble Sort –

Algorithms:



Code:

```
#include <iostream> using namespace std;

void bubbleSort(int arr[], int n)
{    for (int i = 0; i < n - 1; i++)
   {       for (int j = 0; j < n - 1 - i; j++)
      {           if (arr[j] > arr[j + 1])
         {               int temp = arr[j];
                         arr[j] = arr[j + 1];
                         arr[j + 1] = temp;          }
      }
   }
}
 int main()
```

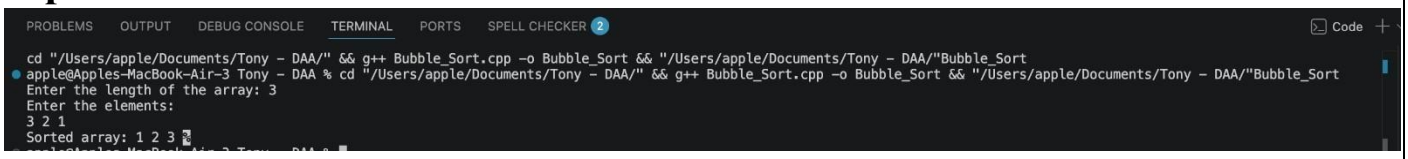
```
{    int arr[] = {5, 3, 8, 4, 2};    int n = 5;

   bubbleSort(arr, n);

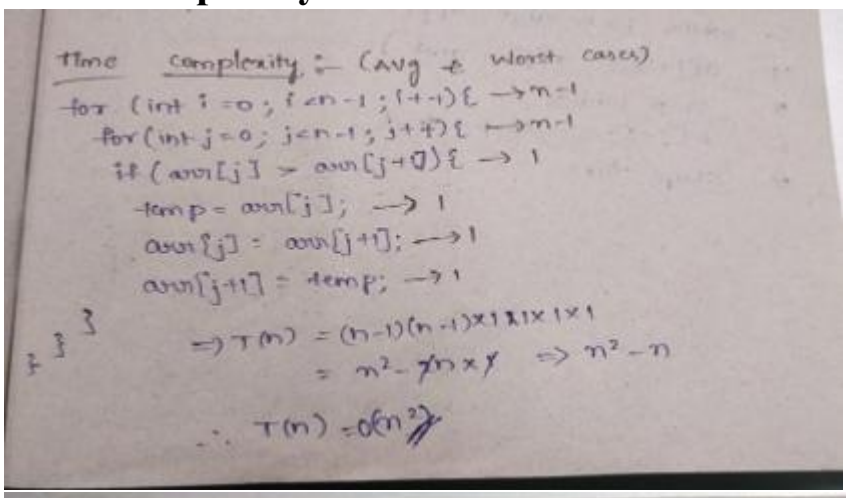
   cout << "Sorted array: ";    for (int i = 0; i < n; i++)
   {       cout << arr[i] << " ";
   }

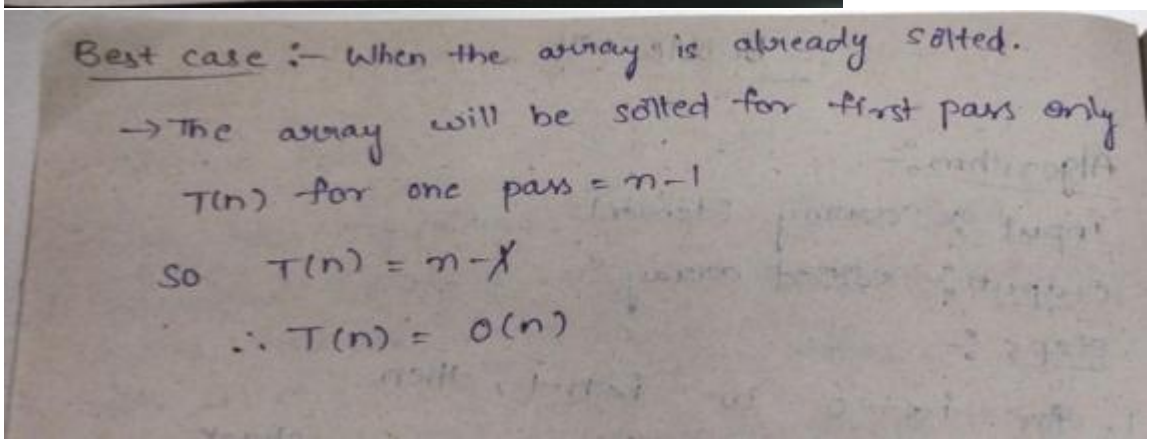
   return 0;
}
```

Output:



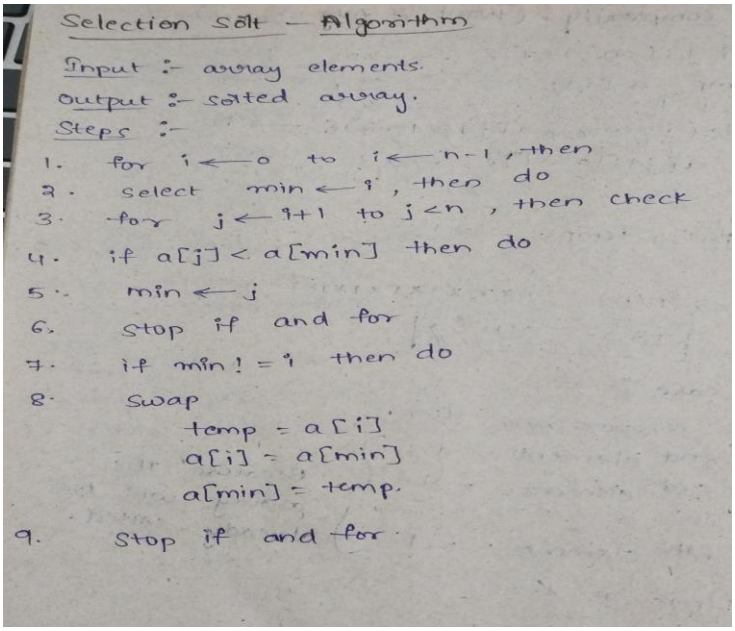
Time Complexity:





Selection Sort–

Algorithms:



Code:

```
#include <iostream> using namespace std;

void selectionSort(int arr[], int n)
{    for (int i = 0; i < n - 1; i++)
   {
      int minIndex = i;

      for (int j = i + 1; j < n; j++)
      {          if (arr[j] < arr[minIndex])              minIndex = j;
      }

      int temp = arr[i];       arr[i] = arr[minIndex];       arr[minIndex] = temp;
   }
}

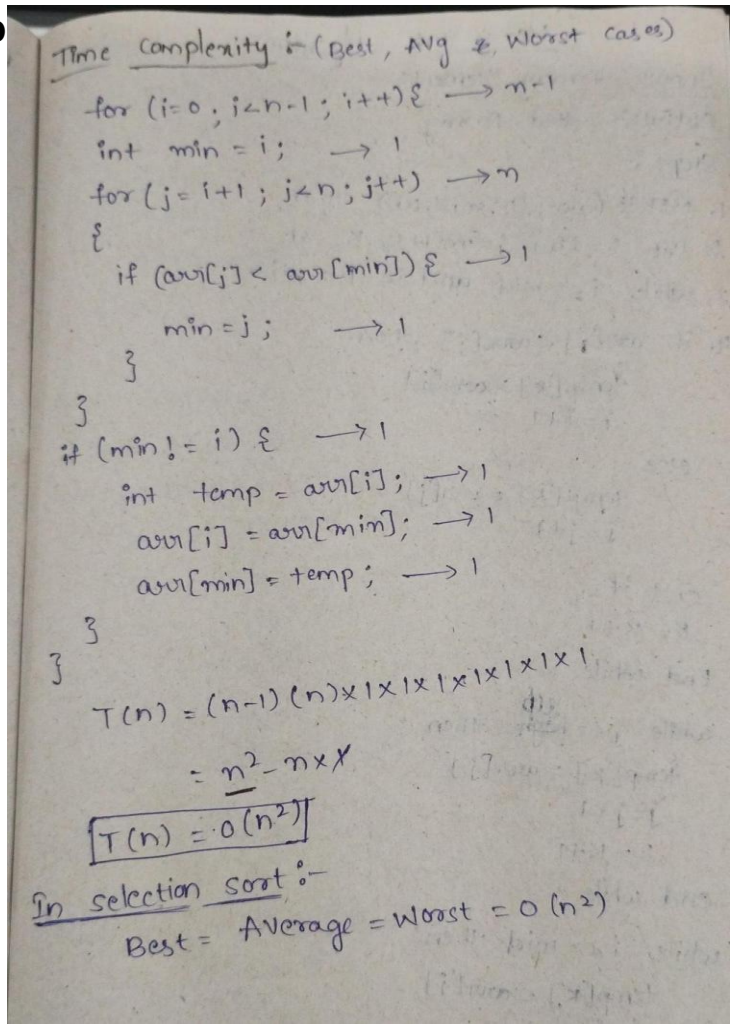
int main() {    int n;
```

```
    cout << "Enter number of elements: ";     cin >> n;
    int* arr = new int[n];
    cout << "Enter " << n << " elements: ";     for (int i = 0; i < n; i++)        cin >> arr[i];
    selectionSort(arr, n);
  "Sorted array: ";     for (int i = 0; i < n; i++)        cout << arr[i] << " ";
    cout << endl;
    delete[] arr;
    return 0;
}
```

Output:

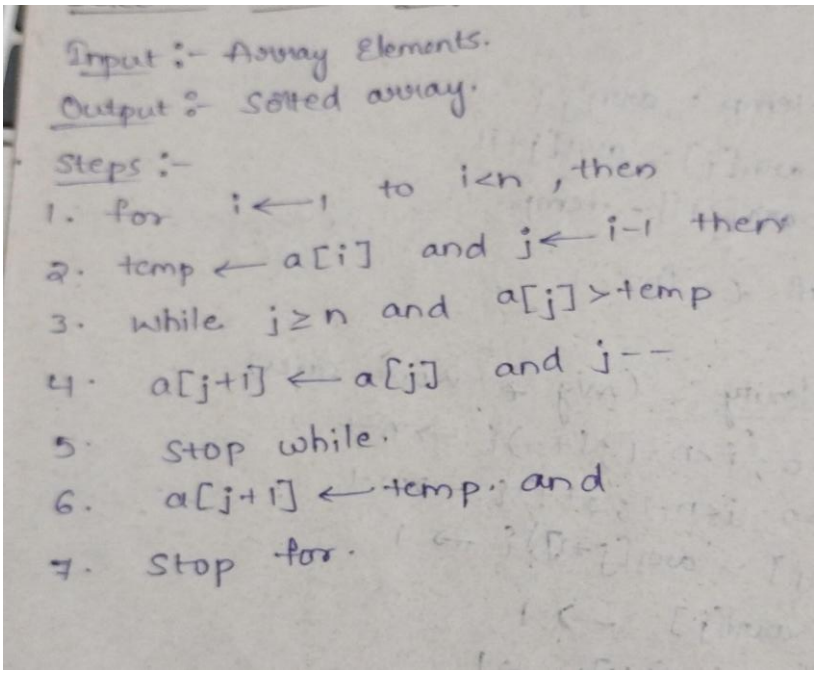
```
apple@Apples-MacBook-Air-3 Tony - DAA % cd "/Users/apple/Documents/Tony - DAA/" && g++ Selection_Sort.cpp -o Selection_Sort && "/Users/apple/Documents/Tony - DAA/"Selection_
Sort
Enter the length of the array: 3
Enter array elements: 3 2 1
Sorted array: 1 2 3
```

Time Comp



Insertion Sort –

Algorithms:



Code:

```
#include <iostream> using namespace std;

void insertionSort(int arr[], int
n) {    for (int i = 1; i < n; i++)
   {       int key = arr[i];       int j = i - 1;

 |
       while (j >= 0 && arr[j] > key)
       {           arr[j + 1] = arr[j];           j--;
       }

       arr[j + 1] = key;
   }
}

int main() {    int n;
```

```
    cout << "Enter number of elements: ";     cin >> n;

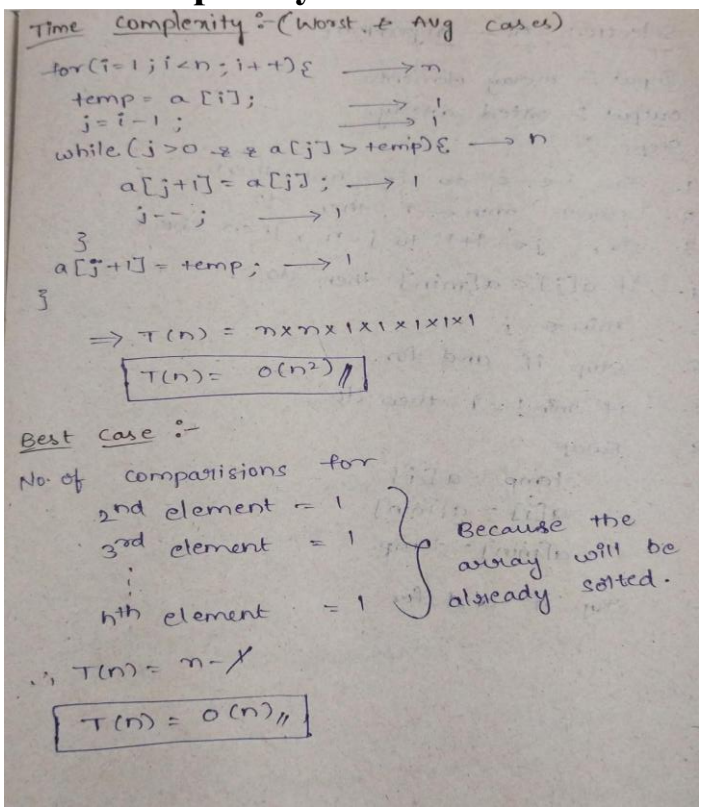
    int* arr = new int[n];

    cout << "Enter " << n << " elements: "     for (int i = 0; i< n; i++)
        cin >> arr[i];     insertionSort(arr, n);     cout << "Sorted array: ";     for (int i = 0; i
< n; i++)         cout << arr[i] << " ";     cout << endl;     delete[] arr;     return 0;
}
```

Output:

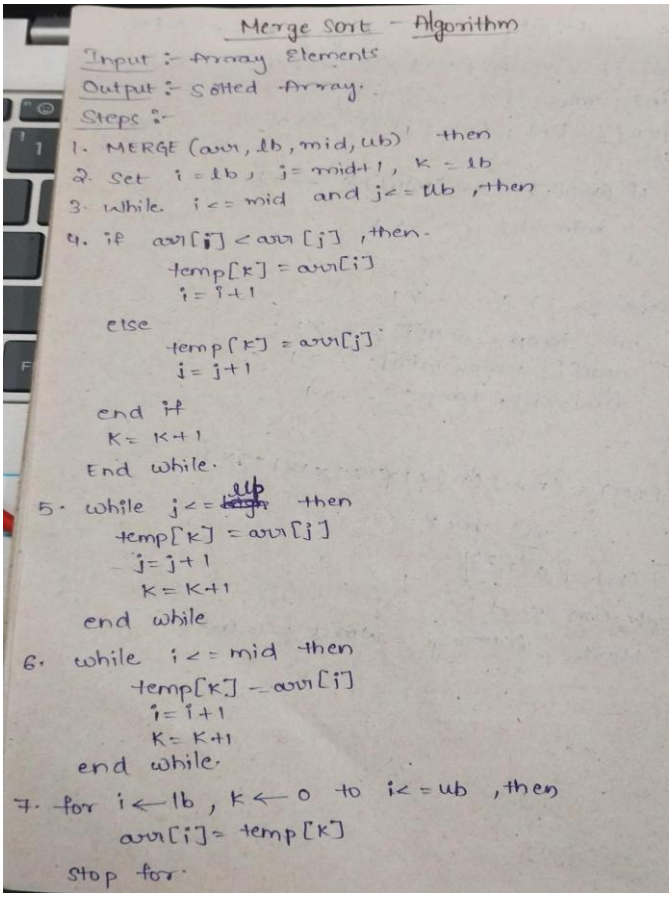
```
apple@Apples-MacBook-Air-3 Tony - DAA % cd "/Users/apple/Documents/Tony - DAA/" && g++ Insertion_Sort.cpp -o Insertion_Sort && "/Users/apple/Documents/Tony - DAA/"Insertion_
Sort
Enter the length of the array: 3
Enter array elements: 3 2 1
Sorted array: 1 2 3
```

Time Complexity:



Merge Sort–

Algorithms:



Merge Sort - Algorithm

Input :- Array Elements
Output :- Sorted Array.
Steps :-

1. MERGE (arr, lb, mid, ub) then
2. Set i = lb, j = mid+1, k = lb
3. while i <= mid and j <= ub, then
4. if arr[i] < arr[j], then
   temp[k] = arr[i]
   i = i+1
   else
   temp[k] = arr[j]
   j = j+1
   end if
   k = k+1
   End while.
5. while j <= ub then
   temp[k] = arr[j]
   j = j+1
   k = k+1
   end while
6. while i <= mid then
   temp[k] = arr[i]
   i = i+1
   k = k+1
   end while.
7. for i ← lb, k ← 0 to i <= ub, then
   arr[i] = temp[k]
   Stop for.

Code:

```
#include <iostream> using namespace std;

void merge(int arr[], int left, int mid, int right)
{    int n1 = mid - left + 1;     int n2 = right - mid;

   int* L = new int[n1];     int* R = new int[n2];

   for (int i = 0; i < n1; i++)
```

```
        L[i] = arr[left + i];

    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2)
    {       if (L[i] <= R[j])
        {           arr[k] = L[i];          i++;        }       else        {           arr[k] = R[j];
j++;        }       k++;    }

    while (i < n1)
    {       arr[k] = L[i];      i++;        k++;    }

    while (j < n2)
    {       arr[k] = R[j];      j++;        k++;    }

    delete[] L;     delete[] R;
}

void mergeSort(int arr[], int left, int right) {     if (left < right)     {       int mid = left +
(right - left) / 2;

        mergeSort(arr, left, mid);      mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);     }
}

int main() {     int n;

    cout << "Enter number of elements: ";     cin >> n;

    int* arr = new int[n];
```

```
    cout << "Enter " << n << " elements: ";     for (int i = 0; i < n; i++)        cin >> arr[i];
mergeSort(arr, 0, n - 1);

    cout << "Sorted array: ";     for (int i = 0; i < n; i++)        cout << arr[i] << " ";

    cout << endl;

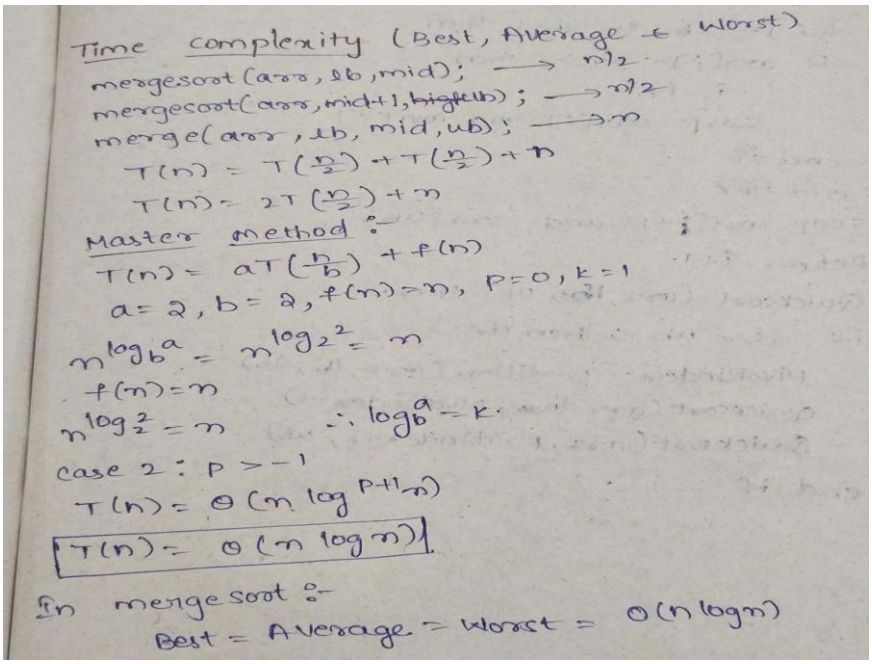
    delete[] arr;

    return 0;
}
```

Output:

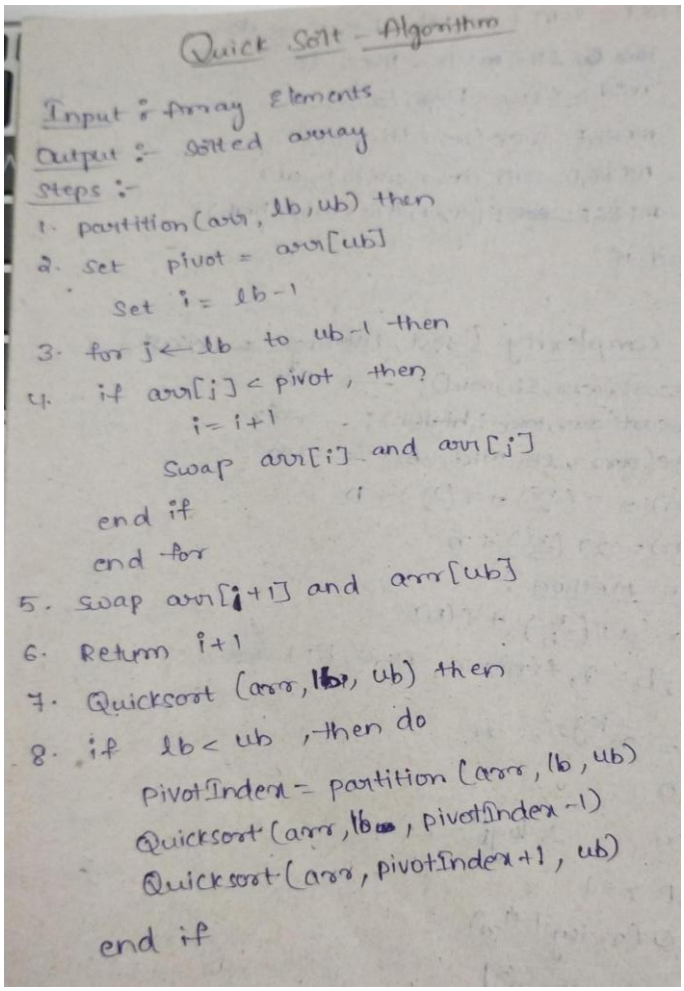
```
apple@Apples-MacBook-Air-3 Tony - DAA % cd "/Users/apple/Documents/Tony - DAA/" && g++ Merge_Sort.cpp -o Merge_Sort && "/Users/apple/Documents/Tony - DAA/"Merge_Sort
Enter number of elements: 3
Enter 3 elements: 3 2 1
Sorted array: 1 2 3
```

Time Complexity:



Quick Sort –

Algorithms:



Code:

```
#include <iostream> using namespace std;

int partition(int arr[], int low, int high)
{    int pivot = arr[high];    int i = low - 1;

   for (int j = low; j < high; j++)
   {
```

```
        if (arr[j] < pivot)
        {           i++;           int temp = arr[i];           arr[i] = arr[j];           arr[j] = temp;
        }
    }

    int temp = arr[i + 1];    arr[i + 1] = arr[high];    arr[high] = temp;

    return i + 1;
}

void quickSort(int arr[], int low, int high)
{    if (low < high)
    {        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);        quickSort(arr, pi + 1, high);
    }
}

int main() {    int n;

    cout << "Enter number of elements: ";    cin >> n;

    int* arr = new int[n];

    cout << "Enter " << n << " elements: ";    for (int i = 0; i < n; i++)
        cin >> arr[i];

    quickSort(arr, 0, n - 1);

    cout << "Sorted array: ";    for (int i = 0; i < n; i++)        cout << arr[i] << " ";

    cout << endl;

    delete[] arr;

    return 0;
```

```
}
```

Output:

```
apple@Apples-MacBook-Air-3 Tony - DAA % cd "/Users/apple/Documents/Tony - DAA/" && g++ Quick_Sort.cpp -o Quick_Sort && "/Users/apple/Documents/Tony - DAA/"Quick_Sort
Enter number of elements: 3
Enter 3 elements: 3 2 1
Sorted array: 1 2 3
```

Time Complexity:

